18
Optimisation des hyperparamètres 101

Automne 2026

Techniques d’apprentissage automatique — 420-C74-SF — gr. 10017
Spécialiste en solutions d’intelligence artificielle — M. Swawola, M.Sc.

NOUS ÉCLAIRONS.
VOUS BRILLEZ.

FORMATION CONTINUE
ET SERVICES AUX ENTREPRISES

Cégep de Sainte-Foy

Sommaire

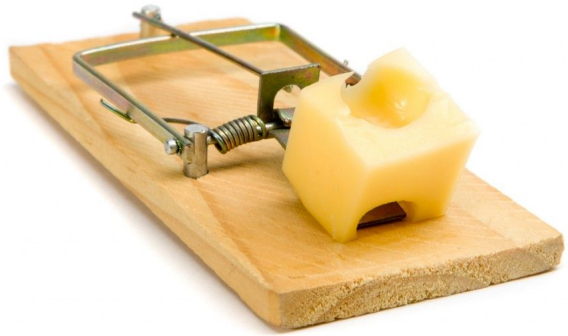
1. Théorème du “No Free Lunch”
2. Méthodes d’optimisation
3. Lectures et références

Sommaire

1. Théorème du “No Free Lunch”
2. Méthodes d’optimisation
3. Lectures et références

Théorème du “No Free Lunch”

Il n’existe pas d’algorithme d’optimisation meilleur que les autres sur l’ensemble des problèmes existants : Il n’y a pas de miracle à attendre ou pas de « repas gratuit » (McCready et Wolpert, 1997)



Les algorithmes miraculeux « boîte noire » n’existent pas, ou plutôt : si tous les algorithmes sont au mieux équivalents, il est impossible de faire mieux qu’une **recherche aléatoire** !

L’objectif est de faire mieux que la recherche aléatoire sur **un problème particulier** (quitte à faire moins sur d’autres problèmes)

Sommaire

1. Théorème du “No Free Lunch”
2. Méthodes d’optimisation
3. Lectures et références

Quelques méthodes d’optimisation

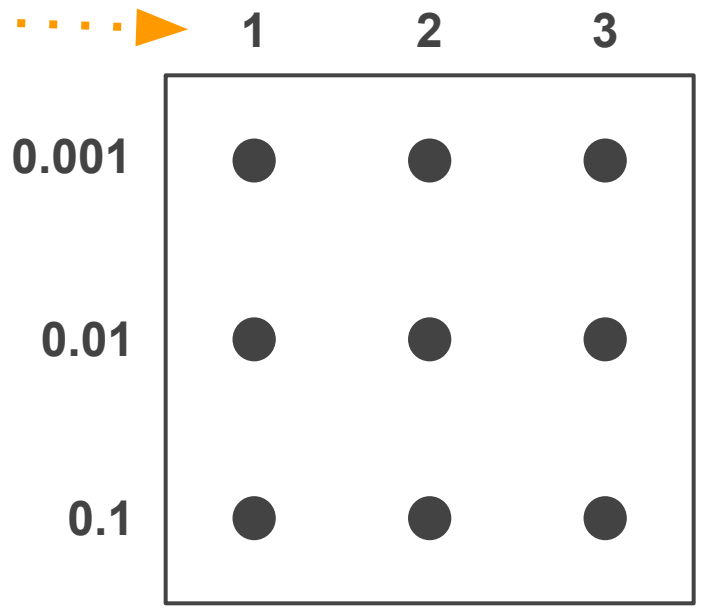
- **Recherche sur grille**
- **Recherche aléatoire**
- **Optimisation séquentielle basée sur un modèle (SMBO)**
- Optimisation Bayésienne
- Optimisation évolutionnaire (algorithmes génétiques)
- Colonies de fourmis (ACO)
- … et bien d’autres !



Recherche sur grille

Exemple: régression logistique avec régularisation L2

- Ordre polynôme
- Paramètre de régularisation



L'ajustement est réalisé sur l'ensemble des combinaisons, ici 3 x 3 = 9 fois

Si validation croisée à 5 plis,
l'ajustement est réalisé 9 x 5 = 45 fois

Recherche sur grille

Exemple: régression logistique avec régularisation L2

- Ordre polynôme
- Paramètre de régularisation

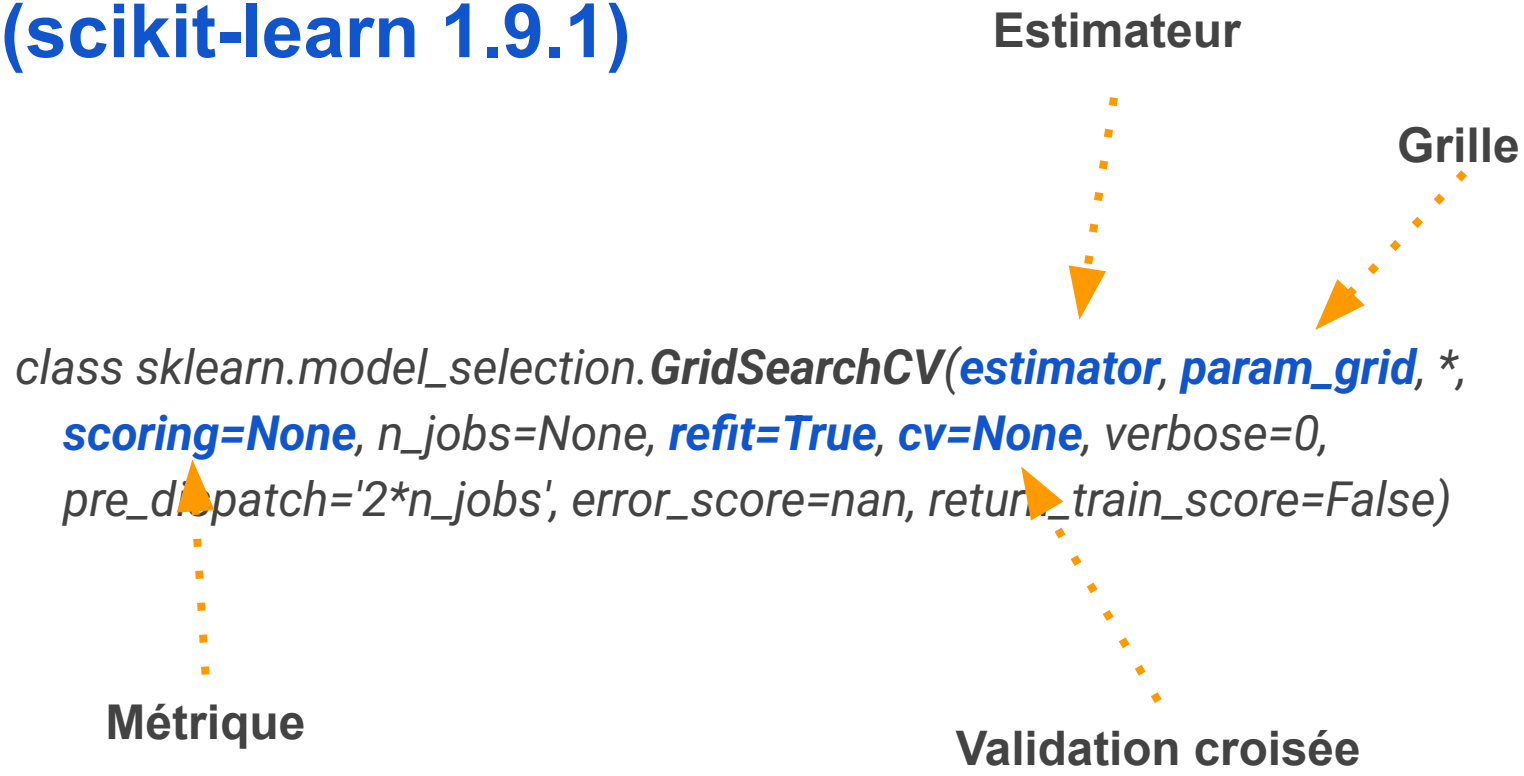
Attention à l'explosion combinatoire !

L'ajustement est réalisé sur l'ensemble des combinaisons, ici 3 x 3 = 9 fois

Si validation croisée à 5 plis,
l'ajustement est réalisé 9 x 5 = 45 fois



GridSearchCV (scikit-learn 1.9.1)

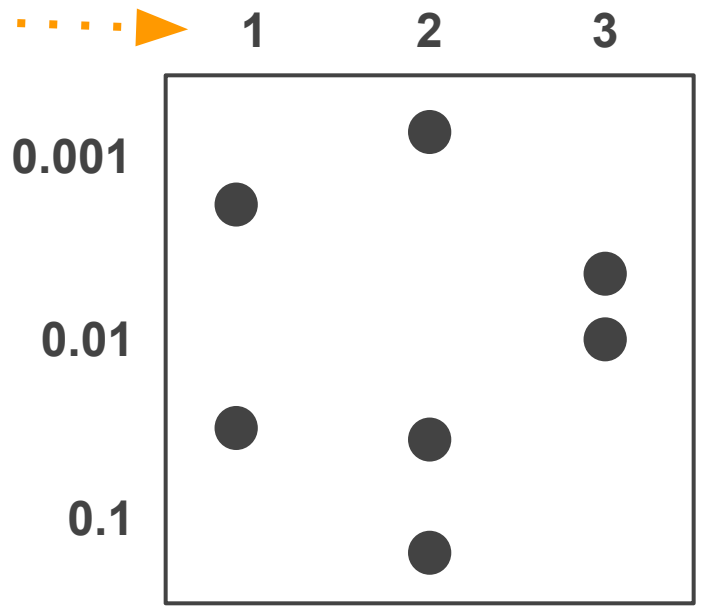


Recherche aléatoire

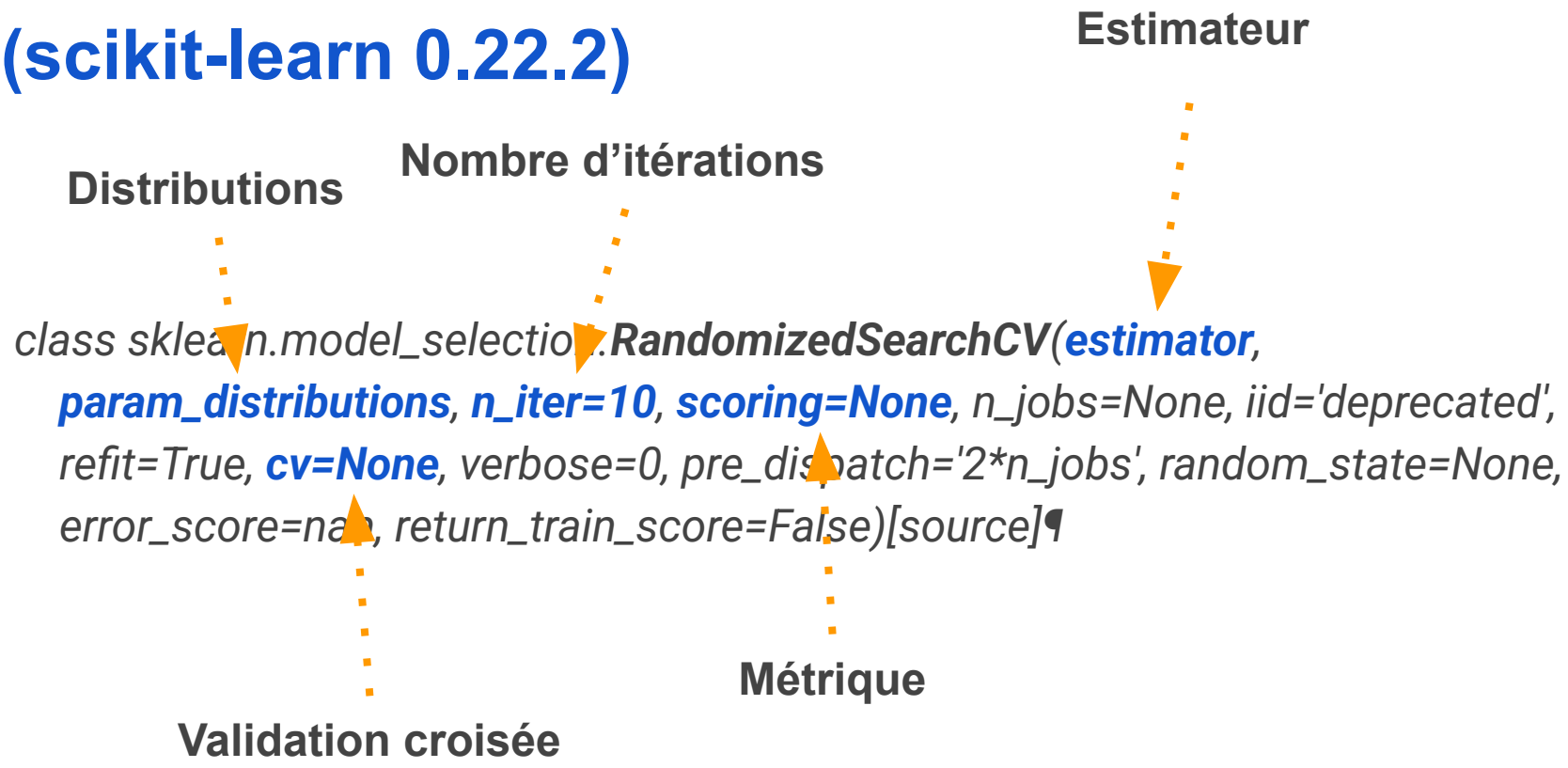
Exemple: régression logistique avec régularisation L2

- Ordre polynôme
- Paramètre de régularisation

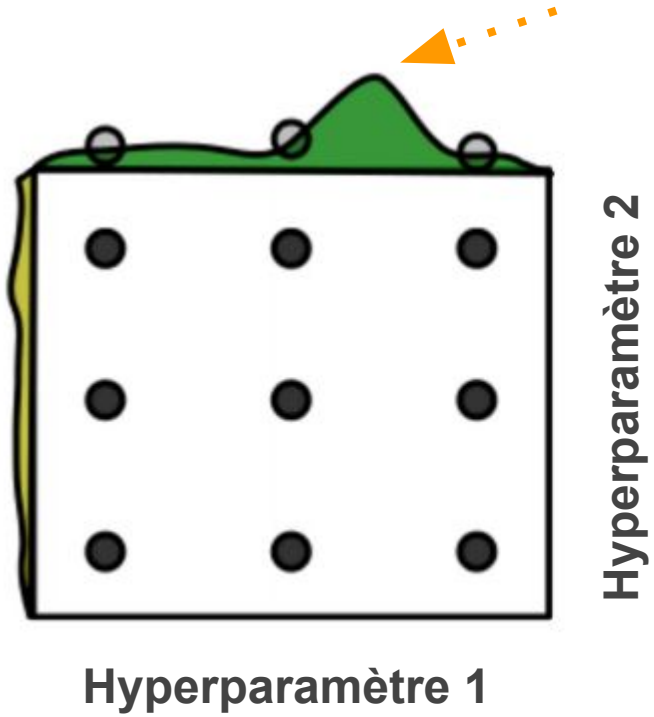
L'ajustement est réalisé un nombre de fois prédéterminé en échantillonnage sur les distributions des paramètres

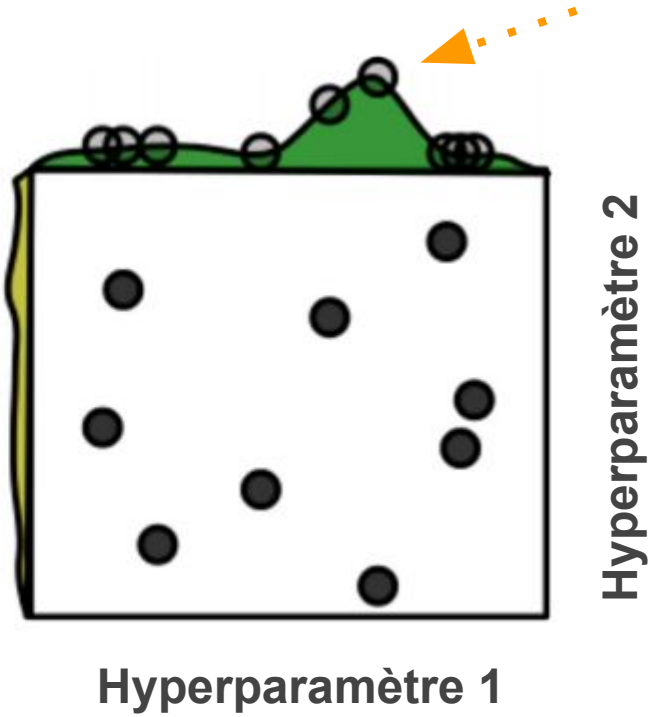


RandomizedSearchCV (scikit-learn 0.22.2)



Recherche aléatoire vs grille





Tree-structured Parzen Estimator

Optimisation séquentielle basée sur un modèle (SMBO)

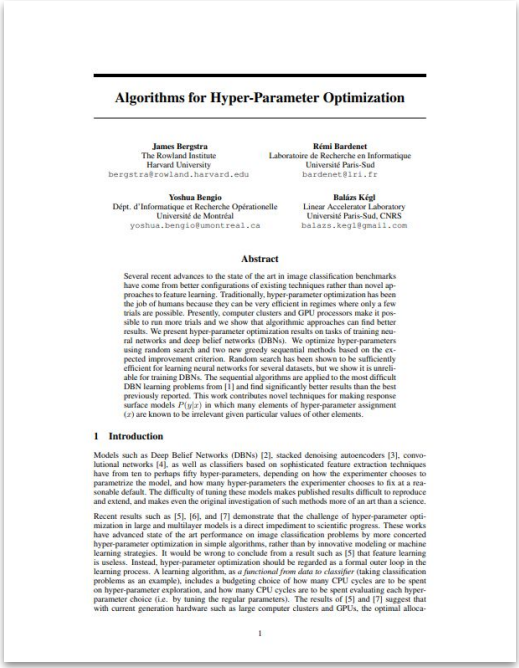
Recherche aléatoire “sur stéroïdes” (basée sur un modèle de probabilité)




```python
from hyperopt import hp
space = hp.choice('classifier_type', [
    {
        'type': 'naive_bayes',
    },
    {
        'type': 'svm',
        'C': hp.lognormal('svm_C', 0, 1),
        'kernel': hp.choice('svm_kernel', [
            {'ktype': 'linear'},
            {'ktype': 'RBF', 'width': hp.lognormal('svm_rbf_width', 0, 1)},
            ]),
    },
    {
        'type': 'dtree',
        'criterion': hp.choice('dtree_criterion', ['gini', 'entropy']),
        'max_depth': hp.choice('dtree_max_depth',
            [None, hp.qlognormal('dtree_max_depth_int', 3, 1, 1)]),
        'min_samples_split': hp.qlognormal('dtree_min_samples_split', 2, 1, 1),
    },
    ])
```

Sommaire

1. Théorème du “No Free Lunch”
2. Méthodes d’optimisation
3. Lectures et références



Algorithms for Hyper-Parameter Optimization

James Bergstra
The Rowland Institute
Harvard University
bergstra@rowland.harvard.edu

Rémi Bardenet
Laboratoire de Recherche en Informatique
Université Paris-Sud
bardenet@lri.fr

Yoshua Bengio
Dépt. d'Informatique et Recherche Opérationelle
Université de Montréal
yoshua.bengio@umontreal.ca

Balázs Kégl
Linear Accelerator Laboratory
Université Paris-Sud, CNRS
balazs.kegl@gmail.com

Abstract

Several recent advances to the state of the art in image classification benchmarks have come from better configurations of existing techniques rather than novel approaches to feature learning. Traditionally, hyper-parameter optimization has been the job of humans because they can be very efficient in regimes where only a few trials are possible. Presently, computer clusters and GPU processors make it possible to run more trials and we show that algorithmic approaches can find better results. We present hyper-parameter optimization results on tasks of training neural networks and deep belief networks (DBNs). We optimize hyper-parameters using random search and two new greedy sequential methods based on the expected improvement criterion. Random search has been shown to be sufficiently efficient for learning neural networks for several datasets, but we show it is unreliable for training DBNs. The sequential algorithms are applied to the most difficult DBN learning problems from [1] and find significantly better results than the best previously reported. This work contributes novel techniques for making response surface models $P(y|x)$ in which many elements of hyper-parameter assignment (x) are known to be irrelevant given particular values of other elements.

1 Introduction

Models such as Deep Belief Networks (DBNs) [2], stacked denoising autoencoders [3], convolutional networks [4], as well as classifiers based on sophisticated feature extraction techniques have from ten to perhaps fifty hyper-parameters, depending on how the experimenter chooses to parametrize the model, and how many hyper-parameters the experimenter chooses to fix at a reasonable default. The difficulty of tuning these models makes published results difficult to reproduce and extend, and makes even the original investigation of such methods more of an art than a science.

Recent results such as [5], [6], and [7] demonstrate that the challenge of hyper-parameter optimization in large and multilayer models is a direct impediment to scientific progress. These works have advanced state of the art performance on image classification problems by more concerted hyper-parameter optimization in simple algorithms, rather than by innovative modeling or machine learning strategies. It would be wrong to conclude from a result such as [5] that feature learning is useless. Instead, hyper-parameter optimization should be regarded as a formal outer loop in the learning process. A learning algorithm, as *a functional from data to classifier* (taking classification problems as an example), includes a budgeting choice of how many CPU cycles are to be spent on hyper-parameter exploration, and how many CPU cycles are to be spent evaluating each hyper-parameter choice (i.e. by tuning the regular parameters). The results of [5] and [7] suggest that with current generation hardware such as large computer clusters and GPUs, the optimal alloca-

James Bergstra, Rémi Bardenet, Yoshua Bengio, Balázs Kégl **“Algorithms for Hyper-Parameter Optimization”**

Références

[1] Optimisation Stochastique Évolutionnaire, Université de Strasbourg (FUN-MOOC)

[2] Cours “Introduction à l’apprentissage automatique”, François Laviolette, CRDM

[3] Hyperopt: Distributed Asynchronous Hyper-parameter Optimization

[4] A Conceptual Explanation of Bayesian Hyperparameter Optimization for Machine Learning